

Documentation: I did not collaborate with any other students on this assignment. I consulted the A4 Office Hours for assistance on multiple problems.

---

## 1.a

Reward engineering can be difficult when real-world action spaces are poorly understood. This can happen when the ultimate objective is not clearly defined or if certain actions have unanticipated consequences. Conflicting objectives can introduce high variability when exploring the action space in the pursuit of a reward function as an agent attempts to navigate tradeoffs.

Specifying and incorrect reward function means that the agent's decision making and the learning objective will be out of sync. Misaligned goals translate to poorer performance when evaluated against the proper objective. In some cases, agents can find loopholes (i.e. reward hacking) that increase performance without solving the task.

In some domains, an incorrect reward function will result in unsafe agent behavior. An example of this would be in military applications where agents are trained to operate unmanned aerial vehicles for specific mission sets. If the agent, acting as the pilot, assigns an excessively high reward to the mission task and sacrifices aerodynamic stability as a result, this could lead to a crash. Since the crash would prohibit the vehicle from accomplishing future missions, this would constitute a misaligned reward structure.

## 1.b

The gymnasium documentation states that "the environment aims to increase the number of independent state and control variables compared to classical control environments." After reading further, the hopper offers a constrained mechanistic action space that allows an agent to explore the interplay between torques, joints, and motion to explore high-order control laws. The reward function is defined by health, forward motion, and anti-cost. The `healthy_reward` term maintains the physical realness of the simulation by ensuring that the hopper is still within the observable window of the simulation and is upright. `forward_reward` gives the hopper a goal by incentivizing forward motion in the x-direction. `ctrl_cost` penalizes the hopper for large actions so that the model finds interesting behavior rather than launching itself with a high torque to complete the task. I would not make changes to this reward function because I think fewer rewards proffer the most interesting results in this case.

## 1.c

The healthy states defined in the Termination section are all except those which find the boundary criteria of the problem. That is, a current position outside the range  $[-100, 100]$ , a height outside  $[0.7, \infty]$ , and a torso angle outside  $[-0.2, 0.2]$ . These boundaries act as safeguards against basic task failures (e.g. falling over) and buggy behaviors like contorting to find a niche reward. A clear advantage of early termination is saving resources on training the agent outside the desired bounds of the problem. This comes at the disadvantage of not discovering a potential unique behavior that could reveal interesting control dynamics.

1.e

From the plots generated by the starter code, it is clear that performance improves with greater training epochs and wall time. However, this relationship shows a logarithmic decay such that only marginal improvements in performance are gained at the top end of the simulation. The standard error on average returns appears moderate doing PPO. This is likely a product of the selection of batch size, learning rate, etc. To obtain a better estimate of the average return achieved by policy optimized with PPO, we could train with more randomized seeds for longer. However, as previously pointed out, this would follow the law of diminishing returns.

## 1.f

The agent did not successfully complete the assigned task under my example policy. In the video, the hopper displays jerky behavior before falling. I was surprised that the policy dictated rapid hinging movements since I thought the training was sufficient to penalize falling or contorting.

1.g

The second rollout successfully completed the assigned task and displayed significantly smoother behavior. I preferred this policy because it controlled the hopper more slowly and deliberately rather than pursuing a jerky, high-cost solution. When relating to human controllers, the latter seemed to take more of a beginner's approach with guaranteed success.

## 2.a

To calculate the gradient of the Bradley-Terry loss function, we can begin by parameterizing terms,

$$\nabla_w \text{loss}(\hat{r}) = -\Sigma_{(\sigma^1, \sigma^2, \mu)} \left[ \mu(1) \nabla_w \log(\hat{P}[\sigma^1 \succ \sigma^2]) + \mu(2) \nabla_w \log(\hat{P}[\sigma^2 \succ \sigma^1]) \right]. \quad (1)$$

Let us examine the individual gradient of the preference term to start. We can expand to say

$$\nabla_w \log(\hat{P}[\sigma^1 \succ \sigma^2]) = \nabla_w (\log \exp(\phi^1) - \log(\exp(\phi^1) + \exp(\phi^2)))$$

where  $\phi^1 = \Sigma_t \phi_w(o_t^1, a_t^1)$  and  $\phi^2 = \Sigma_t \phi_w(o_t^2, a_t^2)$ . Simplifying, we get

$$\begin{aligned} \nabla_w \log(\hat{P}[\sigma^1 \succ \sigma^2]) &= \nabla_w \phi^1 - \nabla_w \log(\exp(\phi^1) + \exp(\phi^2)) \\ &= \nabla_w \phi^1 - \nabla_w \phi^1 \frac{\exp(\phi^1)}{\exp(\phi^1) + \exp(\phi^2)} - \nabla_w \phi^2 \frac{\exp(\phi^2)}{\exp(\phi^1) + \exp(\phi^2)} \\ &= \nabla_w \phi^1 (1 - \hat{P}[\sigma^1 \succ \sigma^2]) - \nabla_w \phi^2 \hat{P}[\sigma^1 \succ \sigma^2]. \end{aligned}$$

We substitute this final expression into (1), as well as its partner term using the same arithmetic for  $\nabla_w \log(\hat{P}[\sigma^2 \succ \sigma^1])$ . Fully expanded, this becomes

$$\nabla_w \text{loss}(\hat{r}) = -\Sigma_{(\sigma^1, \sigma^2, \mu)} \left[ \mu(1) \left( \nabla_w \phi^1 (1 - \hat{P}[\sigma^1 \succ \sigma^2]) - \nabla_w \phi^2 \hat{P}[\sigma^1 \succ \sigma^2] \right) + \mu(2) \left( \nabla_w \phi^2 (1 - \hat{P}[\sigma^2 \succ \sigma^1]) - \nabla_w \phi^1 \hat{P}[\sigma^2 \succ \sigma^1] \right) \right]$$

Lastly, we recognize that  $\hat{P}[\sigma^2 \succ \sigma^1] = 1 - \hat{P}[\sigma^1 \succ \sigma^2]$  to simplify one final time and conclude with the gradient of the loss function

$$\nabla_w \text{loss}(\hat{r}) = -\Sigma_{(\sigma^1, \sigma^2, \mu)} \left[ \nabla_w \phi^1 \left( \mu(1) - \hat{P}[\sigma^1 \succ \sigma^2] \right) + \nabla_w \phi^2 \left( \mu(2) - \hat{P}[\sigma^2 \succ \sigma^1] \right) \right].$$

## 2.b

With the videos only lasting 0.2 seconds, it is very difficult to develop a preference and scrutinize the labels. From the limited viewing window, I would say I agree with the label because the sequence that was not preferred exhibited jerky behavior. This could be a transient effect though that would resolve with more timesteps.



## 2.c

With the 4 additional samples, I agreed with the preference label 3/5 times, or 60%. This is a small sample size so it is not rigorous to conclude anything about the dataset of preferences, especially since my own are not calibrated at an expert level with respect to the hopper task. However, anything lower than total agreement should indicate to us that there is a chance the preference dataset is low quality, and that the RLHF training will suffer as a result.

2.e

The two reward functions appear to correlate based on the results in Figure 2. The learned reward function earns lower overall rewards, but the trendline follows that of the original environment at several iteration milestones. The original result has significantly higher noise. It is possible that the learned reward function is only finding one or two of the true rewards, causing lower overall returns. More importantly, the correlation offered by the results indicates that the learned RLHF model has successfully captured important elements of the true reward, because similar relative values appear along the same trajectory.

## 2.f

The reward function is not generally identifiable. Experimentally, we could explore many more possible trajectories. This would explicitly help us conclude whether only one possible reward function could explain the set of preferences. However, the literature provided in the course notes offer analytical approaches to prove whether a given reward function is identifiable. In this case, we are examining the partial return preference model, where the authors prove that it is not generally identifiable.

2.g

The end of training video showed an agent that was still a bit unstable but generally slower-moving, more deliberate, and more stable than the demonstrations from the dataset. The hopper made a few large actions in the playback that were likely penalized by the true reward but there were few unexpected behaviors compared to the agent trained from scratch. The hopper still seemed to wobble as if the final RLHF agent never found conclusive reward criteria.

### 3.d

The overall performance across the two methods differs a bit, indicating that the training data is not sufficiently rich. Otherwise, I would suspect that DPO and RLHF would be more comparable. The former exhibited faster convergence and better local optima with a similar final reward result. With respect to pros and cons, DPO was faster to run and computationally lightweight in comparison to RLHF. This, paired with faster convergence, indicates to me that DPO is better suited to the control feedback task introduced by the hopper. Since DPO bypasses the reward model structure, it showed that it was less prone to errors at the start of training. This resulted in higher peak rewards than the learned RLHF model. DPO even kept pace with the environment model across the first 10 iterations or so.

3.e

Based on the video, DPO resulted in a faster moving hopper. This likely means that the model prioritized forward motion behavior to reach the final goal early, and improving this ability rapidly without ever negatively penalizing large actions. The RLHF hopper seems a little less coordinated and slower.

## 4.a

I would suspect that the human raters providing pairwise preferences for generic LLM RLHF/DPO would be people with a basic threshold education who are looking for a supplemental income in a setting that offers a short professional growth horizon. This would likely be lower-middle class individuals like students and people from developing countries. In some cases, a company might hire specialized contractors for a highly niche project, but in general I think the preferences would come from lower income, younger professionals. This could promote a number of issues by amplifying the biases of the rater population if they are demographically homogeneous, or similarly limiting diversity and representation. With the case of developing countries where labor laws are different, the labor of the raters could also be exploited.

## 4.b

I think the answer depends on the use case but I am generally partial toward implementing RLHF for medical diagnoses. RLHF is particularly powerful in highly-complex settings with a large amount of human data because the human feedback can act as a strong guide for agent learning with huge state and action spaces. The feedback from experienced medical professionals would likely improve the early learning of particular models much more efficiently than other RL methods. With that said, many disadvantages to RLHF in the medical context also come to mind, such as the variability of rater expertise, rater bias, incorrect rewards/penalties for diagnostic success, data scarcity, and privacy policies. Privacy would impact the labeling of the data informing the machine learning process. Therefore, I think the nature of the dataset and the use case should dictate the implementation of RLHF.



## 4.c

From the provided paper, I was particularly interested to read about the data quality challenges. The issue of the inherent cost/quality tradeoff when collecting human feedback was interesting to consider. In summary, budgeting for quality labeled data and long conversations is a fundamental drawback that will always have an impact on RLHF models. Data diversity can help alleviate the effects of a low-depth dataset, but this is seemingly the root of most RLHF challenges. The article notes that cost constraints push companies using RLHF to cut corners by "freely sourcing data from product users" which would have several ethical concerns. As users develop a greater continuity with an LLM product, for example, the company would be able to model individual feedback mechanisms to reinforce positive responses from individual users, thereby increasing the perceived quality of their product.